Threading and Concurrency

This document explains how the system uses threading to run multiple components concurrently and the considerations involved.

Overview

The Kiosk Sign-In System runs two separate application interfaces (Tkinter kiosk and Flask dashboard) within a single process. It also performs background I/O operations (email sending) without blocking the user interface. All of this is achieved through Python's `threading` module.

Main Architecture: Kiosk + Dashboard

The entry point (`app.py`) builds the Tkinter kiosk app first, then starts the Flask dashboard in a background thread before entering the Tkinter main loop:

```
def _start_dashboard():
    from dashboard.app import app as dashboard_app
    from waitress import serve
    serve(dashboard_app, host="0.0.0.0", port=5000)

if __name__ == "__main__":
    kiosk_app = App()
    dashboard_thread = threading.Thread(target=_start_dashboard, daemon=True)
    dashboard_thread.start()
    kiosk_app.run()
```

The dashboard is served by Waitress rather than Flask's own `app.run(...)` dev server -- Flask's development server explicitly warns it isn't fit for anything beyond local testing (no real concurrency handling, not hardened against slow/malicious clients), which matters now that the dashboard can be reachable over the internet via a reverse tunnel (see 09-authentication-and-security.md). `waitress.serve()` is a blocking call with no special main-thread requirement -- exactly like the `app.run(..., use_reloader=False)` call it replaces -- so it drops into the existing daemon-thread setup unchanged. It also defaults to a small worker-thread pool internally, a mild concurrency improvement over the old single-threaded dev server.

Import Ordering: Build `App()` Before Starting the Dashboard Thread

`App()` is constructed *before* `dashboard_thread.start()` is called, not after. This matters because `App.__init__` calls `database.init_db()`, which (since the dashboard login model was added) does `from werkzeug.security import generate_password_hash` to seed the default dashboard account. That's the first time anything in the process touches `werkzeug`. Meanwhile, `_start_dashboard()` -- running on its own thread -- imports `dashboard.app`, which imports `flask`, which also fully imports `werkzeug`.

If both threads race to import `werkzeug` for the first time at the same moment, they can each observe the other's *partially initialized* submodules -- `werkzeug`'s own package `__init__.py` has legitimate internal cross-references (`werkzeug` → `werkzeug.test` → `werkzeug.utils` → back into `werkzeug.wsgi`) that CPython's import system safely serializes across threads via a per-module import lock. PyInstaller's frozen import machinery doesn't implement that locking as robustly, so under a packaged `.exe` this race can surface as:

```
ImportError: cannot import name 'wrap_file' from partially initialized module 'werkzeug.wsgi' (most like
```

Building `App()` fully on the main thread first means `database.init_db()`'s `werkzeug` import completes -- serially, no race -- before the dashboard thread is even created. By the time the dashboard thread does its own `import flask`, `werkzeug` is already fully loaded and cached in `sys.modules`, so there's nothing left to race. This only matters for a frozen/PyInstaller build; it's invisible when running from source, which is why it wasn't caught until the packaged kiosk `.exe` was tested.

Why a Daemon Thread

The dashboard thread is created with `daemon=True`. Daemon threads are automatically terminated when the main thread exits. This means:

- Closing the kiosk window ends the main thread's `mainloop()`.
- The daemon dashboard thread is immediately killed.
- No cleanup code is needed to shut down Flask.

Without `daemon=True`, the Flask server would keep the process alive even after the kiosk window is closed.

Why Not Multiprocessing

Threads were chosen over processes because:

1. Both components access the same SQLite database file. SQLite handles concurrent access from multiple threads within a process well (especially with WAL mode), but cross-process locking is more complex.
2. A single process is simpler to deploy and manage, especially on a single-purpose kiosk device.
3. The Flask server and Tkinter GUI do not compete for CPU -- Flask is I/O-bound (waiting for requests), and Tkinter is event-driven (waiting for user input).

Why Not Flask's Dev Server

Flask's built-in `app.run()` has a debug/reloader mode that starts a subprocess and watches files for changes -- appropriate for local development, but it would conflict with (and potentially kill) the Tkinter main loop if ever enabled in this embedded setup. Waitress has no such reloader or debug subprocess to worry about; it just serves the WSGI app directly, which is both simpler and correct for this threading model.

Background I/O Operations

Email Notifications

When a member is signed in or out, an email notification is sent asynchronously:

```
def send_notification(member, direction, timestamp):
   threading.Thread(
       target=_send, args=(member, direction, timestamp), daemon=True
   ).start()
```

A new thread is spawned for each notification. This is appropriate because:

- SMTP operations can take several seconds (DNS lookup, TLS handshake, authentication, message delivery).
- The kiosk UI must remain responsive during this time.
- Email sending is fire-and-forget -- we do not need the result.

The daemon flag ensures that pending email threads do not prevent the application from exiting.

SQLite Thread Safety

SQLite supports multiple concurrent readers and a single writer. The system enables WAL (Write-Ahead Logging) mode on every connection:

```
conn.execute("PRAGMA journal_mode=WAL")
```

WAL mode allows: - Multiple threads to read simultaneously. - A single thread to write while others read. - No blocking between the kiosk thread and the dashboard thread for read operations.

Each function in `database.py` opens its own connection (`get_conn()`), performs the operation, and closes it. This avoids sharing connection objects between threads, which SQLite does not support safely in all configurations.

Tkinter and the GIL

Python's Global Interpreter Lock (GIL) ensures that only one thread executes Python bytecode at a time. This has two implications for this system:

> **CPU-bound work**: The GIL prevents true parallel execution of Python code. However, this system's background threads are I/O-bound (waiting for network responses, database writes), so the GIL is released during those waits, allowing other threads to run.
>
> **Tkinter thread safety**: Tkinter is not thread-safe -- widgets must only be modified from the main thread. Background threads should use `widget.after(0, callback)` to schedule UI updates on the main thread's event loop. The email notifier avoids this issue entirely by not touching the UI.

Virtual Keyboard Focus Management

The virtual keyboard uses a threading-adjacent technique -- delayed callbacks with `widget.after()`:

```
def _on_focus_out(self, event):
    self._pending_detach = self.root.after(400, self._detach_if_not_entry)
```

When focus leaves an Entry widget, the keyboard detach is delayed by 400ms. If focus moves to another Entry within that window, the pending detach is cancelled. This prevents the keyboard from flickering when the user taps between input fields.

This is not threading in the traditional sense, but it uses the same principle of deferred execution -- scheduling work to happen later based on the current state.

Summary of Threading Use

Component	Thread Type	Purpose
Tkinter mainloop	Main thread	Kiosk GUI event loop
Flask server	Daemon	Web dashboard HTTP server
Email notifications	Daemon	SMTP email sending